

step: 3.4

plan: ../PLAN-MASTER.md

agent: tech-lead

status: done

completed_at: 2026-07-23 09:24

STEP 3.4 — Tech Lead review CCU client + UI + integration

Input nhận

Từ STEP-3.3 Handoff Log — CcuClient (ceffc627), CcuUI (17e94016), integration IPGSUseCam (eef68016).

Nhiệm vụ

Review toàn bộ CCU-side + tự fix bug rõ ràng nhỏ; build sạch cả 3 project; xác nhận không đùng IPGS.Object/KztekComponentAvalonia; giao demo cho UXR/QA.

Definition of Done

- [x] `dotnet build IPGSUseCam.csproj` PASS 0 Error (transitively build CcuClient + CcuUI). Warnings 17 (pre-existing, không mới).
- [x] Scale toạ độ mouse tính đúng — `MapToZcuCoords` xử lý letterbox: so sánh `controlAspect` vs `imageAspect`, tính `imageW/imageH` theo `Stretch.Uniform`, offset căn giữa `(controlW-imageW)/2`, clamp về `[0, imageW-1]` trước khi chia tỷ lệ về `screenW/screenH`. Không lệch offset khi ảnh không full-fill.
- [x] Memory leak `FrameReceived` — **PHÁT HIỆN VÀ TỰ FIX**: `CurrentFrame = wb` gán bitmap mới mỗi frame nhưng KHÔNG dispose bản cũ → `WriteableBitmap` giữ native pixel buffer (~8 MB × 15 fps ≈ 120 MB/s allocation, GC không kịp). Sửa: `Dispatcher.UIThread.Post(() => { var old = CurrentFrame; CurrentFrame = wb; old?.Dispose(); })` — gán mới trước rồi dispose bản cũ trên UI thread (tránh race với render). Bổ sung `CurrentFrame?.Dispose()` cuối `Dispose()`. Rebuild PASS. `SKBitmap` (skBmp và converted) đã đúng — `using + mustDispose` trong finally.
- [x] `RemoteScreenControl` là UserControl riêng (`Views/RemoteScreenControl.axaml + .axaml.cs`), Window chỉ host + xử lý toolbar. Tuân thủ §20.4.
- [x] IPGSUseCam chỉ có: `<ProjectReference>` tới CcuUI (+1 dòng csproj), menu item mới trong `MainWindow.axaml` (+4 dòng), handler `OnRemoteZcuMenuClick` mở `RemoteZcuConnectionDialog` (+7 dòng `MainWindow.axaml.cs`), dialog mới `RemoteZcuConnectionDialog.axaml/.axaml.cs` (+99 dòng, hoàn toàn mới). `git diff --stat 6dbda8a3..HEAD -- IPGSUseCam/` xác nhận tổng cộng 5 file, 110+/1-, không sửa business logic cũ.
- [x] IPGS.Object/ và KztekComponentAvalonia/ — không có plan-commit nào chậm; commit 0b88e61a (Linux .deb + SkiaSharp→Avalonia camera + KzMenu theme) không thuộc plan này, là công việc song song của user.

- [x] Ghi lesson gotcha [avalonia/avalonia-writeablebitmap-binding-not-disposed-leak.md](#) + cập nhật INDEX.md + LESSONS-LOG.md (entry 82).

Đã làm

1. Build `IPGSUseCam.csproj` (bao trùm cả CcuClient + CcuUI qua ProjectReference) — PASS 0 Error, 17 Warnings pre-existing.
2. Đọc `RemoteScreenViewModel.cs`:
 - Verify `MapToZcuCoords` — công thức letterbox-aware đúng, có clamp biên; throttle 60 Hz + delta ≥ 2 px cho MOUSE_MOVE khớp TDD §14.
 - Phát hiện leak: `CurrentFrame` gán new `WriteableBitmap` mỗi frame, bản cũ không Dispose → chỉ chờ finalizer.
 - Fix leak (2 chỗ: `OnFrameReceived` UI-thread post + `Dispose()`). Rebuild PASS.
3. Đọc `RemoteScreenControl.axaml.cs` + `RemoteScreenWindow.axaml.cs`:
 - UserControl tách file riêng; Window chỉ host UserControl + xử lý `OnOpened/OnClosed/OnDisconnectClick`. §20.4 OK.
 - Pointer handlers ủy quyền tọa độ về ViewModel — testable, đúng separation.
4. `git diff --stat` xác nhận phạm vi thay đổi ở IPGSUseCam đúng như 3.3, IPGS.Object và KztekComponentAvalonia không bị plan này đụng.
5. Ghi lesson + convert DOCX (PDF fail do docx2pdf/Word — không block); cập nhật INDEX.md + LESSONS-LOG.md entry 82.

Artifact

- `IPGS.RemoteControl.CcuUI/ViewModels/RemoteScreenViewModel.cs` — fix leak (`OnFrameReceived` + `Dispose`)
- `C:\Users\nguye\.claude\lessons\avalonia\avalonia-writeablebitmap-binding-not-disposed-leak.md` (+ .docx)
- `C:\Users\nguye\.claude\lessons\INDEX.md` — thêm entry
- `C:\Users\nguye\.claude\lessons\LESSONS-LOG.md` — entry 82 (+ .docx đã regen)

Quyết định quan trọng

1. **Fix leak in-place, không escalate:** bug rõ ràng thuộc "vấn đề nhỏ có thể tự sửa" theo yêu cầu bước 3.4; không đổi API/kiến trúc, chỉ thêm dispose. Reviewer/Author (cùng session review) OK vì fix ≤ 15 dòng.
2. **Không tối ưu double-buffer WriteableBitmap ở v1:** để lại như v1.1 optimization theo comment trong code. Fix leak là đủ cho demo và stress test.
3. **Không sửa IPGSUseCam thêm:** đã có RemoteZcuConnectionDialog hoạt động; UI chi tiết (validate, remember last, đọc từ StaticPool.ZCU khi có field IP) để UX/feedback rồi mới bổ sung.

Handoff Log — bước sau cần biết

- **Đã làm:** Review CCU-side PASS; tự fix bug memory leak `WriteableBitmap` trong `RemoteScreenViewModel.OnFrameReceived + Dispose()`; build sạch cả 3 project (CcuClient, CcuUI, IPGSUseCam); ràng buộc cứng (không đụng IPGS.Object/KztekComponentAvalonia/business logic cũ) tôn trọng đầy đủ; ghi lesson gotcha.

- **File/module đã đọc hoặc đổi:**
 - Đọc: `IPGS.RemoteControl.CcuUI/ViewModels/RemoteScreenViewModel.cs`,
`Views/RemoteScreenControl.axaml.cs`, `Views/RemoteScreenWindow.axaml.cs`.
 - Sửa: `IPGS.RemoteControl.CcuUI/ViewModels/RemoteScreenViewModel.cs` (2 chỗ dispose bitmap cũ).
- **Quyết định quan trọng:** fix leak in-place; hoãn tối ưu double-buffer sang v1.1.
- **Bước sau cần biết — cách chạy demo end-to-end:**

A. Chuẩn bị ZCU-side (Linux):

1. Cần Ubuntu 22.04 với X11 (WSL2 KHÔNG chạy được vì `XTest/XShm` cần X server thật, WSL2 default là Wayland/không có display). Dùng **máy Linux thật** hoặc VM (VirtualBox/VMware) với `Ubuntu 22.04 Desktop`.
2. Copy folder `IPGS.RemoteControl.ZcuAgent/` sang máy Linux.
3. Cài .NET 8 SDK trên Linux (`sudo apt install dotnet-sdk-8.0`).
4. Build: `cd IPGS.RemoteControl.ZcuAgent && dotnet build -c Release`.
5. Chỉnh `appsettings.json`: `"AuthToken": "demo-token-1234", "ListenPort": 17600, "AllowedIPs": ["0.0.0.0/0"]` (mở hết cho demo, siết lại sau).
6. Chạy: `dotnet run -c Release` — agent listen 0.0.0.0:17600.
7. Kiểm tra IP máy Linux: `ip addr show | grep inet`.

B. Chuẩn bị CCU-side (Windows, iPGSUseCam):

1. Build `iPGSUseCam` như bình thường (đã có `<ProjectReference>` tới `CcuUI`, tự pull vào).
2. Chạy `iPGSUseCam`.
3. Menu **"Hệ thống"** → **"Điều khiển ZCU từ xa..."** → `RemoteZcuConnectionDialog` mở ra.
4. Nhập: **IP** = IP máy Linux (VD `192.168.1.50`), **Port** = `17600`, **Token** = `demo-token-1234` (khớp với `appsettings.json` ZCU).
5. Click **Kết nối** → `RemoteScreenWindow` mở ra, tự connect. Trạng thái toolbar: Chưa kết nối (xám) → Đang kết nối (cam) → Đang xác thực (cam) → Đã kết nối (xanh lá).
6. Màn hình ZCU stream qua ở ~10–15 fps. Click chuột trong vùng ảnh → ZCU nhận và inject qua `XTest` (verify: mở terminal trên ZCU và click vào đó, gõ phím không có trong sprint này — chỉ mouse).
7. Close window → `OnClosed` gọi `DisconnectAsync + Dispose` — ZCU thấy client disconnect gracefully.

C. Điểm cần UXR (Bước 4.1) kiểm tra:

- Layout `RemoteScreenControl` (letterbox căn giữa, ảnh không méo).
- Toolbar trạng thái + màu dot (xám/cam/xanh/đỏ) rõ ràng, có nút Ngắt kết nối.
- Error banner khi Faulted (`AUTH_FAIL` → "Token không hợp lệ..."; timeout/reconnect fail → "Không thể kết nối sau nhiều lần thử...").
- Dialog `RemoteZcuConnectionDialog` UX (validate IP/port/token, Cancel/OK).

D. Điểm cần QA (Bước 4.2) kiểm tra:

- Happy path connect → stream → disconnect.
- Sai token → nhận `AUTH_FAIL`, KHÔNG auto-reconnect (đúng TDD §7).
- Sai IP/port → reconnect có backoff, sau N lần bỏ (`Max reconnect...`).

- Mouse click trong vùng ảnh → ZCU inject đúng toạ độ (test bằng cách click vào 1 button trên desktop ZCU, verify button được click).
- Đóng window đột ngột (Alt+F4) → không leak, ZCU nhận disconnect.
- **RAM stability**: sau fix leak, RAM phải ổn định sau ~30s streaming (không tăng đều nữa).
- PING/PONG heartbeat (30s) — nếu 2 PONG timeout liên tiếp → client auto-reconnect.

Commit

- Hash: [chưa commit — theo yêu cầu "KHÔNG tự commit/push"]
- Đã push: không

Status icons: ☐ Todo |  In Progress |  Done |  Blocked |  Skipped